

---

# Scrabble – Java

**Rapport final**

CODOGNAN Thomas, RAZE Erwan, KAHLOUL Aïssa, BLOTHIAUX Yanis, BETTCHER Léonard

---

8 avril 2026

**Table des matières**

# 1 Historique et Nature du Sujet

## 1.1 Historique du sujet

En 1931, l'architecte au chômage Alfred Mosher Butts decide de creer un jeu combinant chance et reflexion. En analysant minutieusement la frequence des lettres a la une du *New York Times*, il elabore un systeme de valeurs specifiques pour chaque jeton, methode qui reste inchangee dans les editions modernes [1]. Initialement baptise *Lexiko*, puis *Criss-Cross Words*, le jeu essuie les refus de nombreux fabricants avant qu'un homme d'affaires, James Brunot, ne rachete les droits en 1948 pour lui donner son nom definitif : Scrabble [2].

Le succes mondial du jeu doit beaucoup a un coup de chance marketing. Dans les annees 1950, Jack Straus, president du grand magasin Macy's a New York, decouvre le jeu pendant ses vacances et s'etonne de ne pas le trouver dans ses rayons. Il passe une commande massive, propulsant le Scrabble au rang de phenomene culturel [2]. Aujourd'hui, avec plus de 150 millions d'exemplaires vendus dans 121 pays et des competitions internationales organisees par la World English-Language Scrabble Players Association (WESPA) et la Federation Internationale de Scrabble Francophone (FISF), il est devenu le jeu de lettres le plus populaire de la planete [3].

Des les annees 1980 et 1990, les premiers logiciels de jeu contre l'ordinateur font leur apparition. L'explosion des plateformes en ligne, notamment l'Internet Scrabble Club (ISC), puis des applications mobiles comme *Scrabble Go* ont elargi considerablement la communaute de joueurs. Au-dela du divertissement, l'informatique a revolutionne la competition de haut niveau grace a des outils d'analyse strategique comme Quackle [4], devenu la reference mondiale pour l'entrainement des joueurs experts. Aujourd'hui, de nombreux projets open source et applications multiplateformes temoignent de la vitalite de l'ecosysteme logiciel autour du Scrabble.

**Sources :** [1] Butts, A.M. (1931), archives de conception du jeu. [2] Fatsis, S. (2001), *Word Freak*, Houghton Mifflin. [3] FISF, Regles officielles du Scrabble francophone, edition 2023. [4] Sheppard, B. et al., *Quackle Scrabble AI*, Computer Scrabble Project, 2006.

## 1.2 Nature et enjeux du jeu

Le Scrabble est un jeu de lettres a information imparfaite (les tirages adverses sont inconnus) et stochastique (soumis a l'alea de la pioche). L'objectif est de maximiser son score sur une grille standard de  $15 \times 15$  cases en y plaçant des mots extraits d'un dictionnaire de reference (ODS en francais, TWL/SOWPODS en anglais).

Le jeu repose sur trois piliers fondamentaux :

1. **Logique spatiale** Tout mot pose doit respecter deux contraintes simultanees : la connexite (chaque nouveau mot doit s'appuyer sur au moins une lettre deja presente sur le plateau, et toutes les lettres posees en un meme tour doivent etre alignees et contigues) et l'orthogonalite (toute collision avec des lettres adjacentes doit former un mot valide dans le dictionnaire de reference).

*Exemple* : si le mot FEU est pose horizontalement, un joueur peut placer la lettre R au-dessus du E pour former RE verticalement, a condition de poser simultanement un mot horizontal valide passant par ce R, comme RATS.

2. **Logique mathematique** Le score d'un coup se calcule en additionnant la valeur de chaque lettre posee, en appliquant d'abord les multiplicateurs de lettres (case lettre compte double/triple), puis les multiplicateurs de mots (case mot compte double/triple). Un joueur qui utilise ses 7 lettres en un seul coup remporte un bonus de 50 points, appele Scrabble.

*Exemple* : au premier tour, un joueur pose AXE avec le X sur la case centrale H8 (case mot  $\times 2$ ). La valeur brute est  $A(1) + X(8) + E(1) = 10$ , doublee a 20 points. Si la case centrale etait une case lettre  $\times 2$  sous le X, on calculerait  $A(1) + X(16) + E(1) = 18$ , puis on appliquerait le multiplicateur de mot.

3. **Logique linguistique** Chaque mot pose, y compris les mots formes par collision, doit appartenir au dictionnaire de reference choisi pour la partie. Tout mot invalide entraine l'annulation du coup et, selon les regles, une penalite de score.

**Deroulement d'une partie** La partie debute obligatoirement par un mot d'au moins deux lettres couvrant la case centrale (H8), dont le score est automatiquement double. A chaque tour, le joueur peut : poser un mot, echanger tout ou partie de ses jetons contre des lettres de la pioche (si le sac contient au moins 7 lettres), ou passer son tour. La partie s'acheve lorsque le sac est vide et qu'un joueur a epuise son chevalet, ou apres trois tours consecutifs sans score. Les lettres restant sur les chevalets sont deduites du score de chaque joueur, et leur valeur totale est ajoutee au score du joueur ayant vide son chevalet en premier.

*Exemple de fin de partie* : le joueur A termine avec 300 points. Le joueur B possede encore un W (10 pts) et un X (10 pts) : son score final est 280 pts, et le score de A monte a 320 pts.

## 1.3 Complexite du jeu et defis algorithmiques

Il est essentiel de distinguer la complexite intrinseque du jeu de celle des algorithmes qui cherchent a le resoudre.

**Espace des etats** Le nombre de positions legales sur un plateau  $15 \times 15$  est estime a environ  $10^{30}$ . Si ce chiffre reste inferieur a celui des echecs ( $\sim 10^{43}$ ), l'incertitude propre au Scrabble, due aux tirages caches de l'adversaire et a l'alea de la pioche, en fait un probleme fondamentalement different : il releve de la theorie des jeux a information imparfaite, contrairement aux echecs ou aux dames.

**Explosion combinatoire a chaque coup** Meme pour un seul tour, la generation de tous les coups legaux est un probleme non trivial. Pour un chevalet de 7 lettres et un dictionnaire de type ODS ( $\sim 380\,000$  mots), il faut explorer toutes les combinaisons de lettres (sous-ensembles du chevalet), toutes les positions et orientations possibles sur la grille, et verifier simultanement la validite de tous les mots formes par collision. Le nombre de placements theoriques au premier tour est deja de l'ordre de plusieurs millions [5].

**La qualite du reliquat** Un des defis strategiques majeurs, et algorithmiques, est que maximiser le score immediat n'est pas toujours optimal. Un bon moteur de jeu doit evaluer la qualite du reliquat (les lettres conservees apres le coup), car des lettres polyvalentes comme les voyelles courantes ou le S offrent de meilleures perspectives pour les tours suivants. A l'inverse, conserver des lettres difficiles comme le W ou le K penalise les coups futurs.

**La gestion des cases bonus** Un autre defi crucial est l'obstruction : ouvrir une case bonus (mot  $\times 3$  ou lettre  $\times 3$  en bord de grille) pour l'adversaire peut lui offrir un avantage considerable. L'IA doit donc anticiper non seulement son propre score, mais aussi les opportunités qu'elle laisse a son adversaire.

**Conclusion sur la difficulte** Ces contraintes font du Scrabble un probleme d'optimisation combinatoire sous incertitude, a la frontiere entre la recherche arborescente, la linguistique computationnelle et la theorie des jeux stochastiques.

**Source :** [5] Appel, A.W. & Jacobson, G.J. (1988), *The World's Fastest Scrabble Program*, *Communications of the ACM*.

## 1.4 Existant algorithmique

L'etat de l'art repose sur des travaux academiques fondateurs et des logiciels eprouves.

**Structures de donnees pour le dictionnaire** La generation rapide de coups legaux repose sur des automates finis representant le dictionnaire :

- le DAWG (*Directed Acyclic Word Graph*), propose par Appel & Jacobson (1988) [5], compresse le dictionnaire en un graphe oriente acyclique permettant une verification en  $O(n)$  de la validite d'un mot;
- le GADDAG, introduit par Steven Gordon (1994) [6], est une extension permettant de generer directement tous les mots pouvant s'ancrer sur une lettre donnee du plateau, accelerant considerablement la recherche des coups legaux.

**Logiciels de reference** Quackle [4] est le standard academique et competitif pour l'analyse strategique. Il utilise des simulations de type Monte-Carlo pour estimer l'esperance de gain de chaque coup sur plusieurs tours a venir, en simulant des milliers de parties possibles.

Elise et Maven sont d'autres moteurs historiques ayant contribue a l'etablissement des meilleures pratiques en IA Scrabble.

**Plateformes de jeu en ligne** L'ISC (*Internet Scrabble Club*) demeure la reference pour la pratique competitive, garantissant une application stricte des regles internationales. Des plateformes plus recentes comme Woogles.io proposent egalement des outils d'analyse integres.

**Open source** De nombreux projets open source (notamment en Python, Java et C++) sont disponibles sur des plateformes comme GitHub, rendant accessible l'implementation d'un moteur de jeu a partir des algorithmes decrits dans la litterature.

**Source :** [6] Gordon, S. (1994), *A Faster Scrabble Move Generation Algorithm*, *Software—Practice and Experience*.

## 2 Architecture du projet

Le projet est organise en couches fonctionnelles strictement separees selon le patron MVC. Chaque package porte une responsabilite unique, avec des dependances majoritairement descendantes.

### 2.1 Decoupage en modules

**Point d'entree** La classe `App` initialise le contexte d'execution : detection de la langue via `I18n`, chargement de la configuration `.scrabblerc` via `ConfigLoader`, puis delegation au mode d'affichage selon les options `commons-cli`. `App` expose des `@FunctionalInterface` (`CliLauncherHandler`, `GuiLauncherHandler`, `ExitHandler`) pour decoupler le point d'entree des implementations concretes et faciliter les tests.

**Couche controleur** Le package `controller` contient `GameController` (logique applicative principale) et `GameControllerAux` (orchestration CLI extraite pour limiter la taille de la classe principale). `GameController` maintient les references au modele (`Game`, `Gaddag`), a la vue (`UserInterface`) et aux parametres de lancement

(mode blitz, super-scrabble, IA). Il depend egalement de sous-contrôleurs specialises : `builders/` pour la construction interactive des coups, `config/` pour les snapshots de configuration, `dictionary/` pour le chargement du dictionnaire, et `network/` pour les commandes reseau CLI.

**Couche metier — core** Le package `model.core` regroupe les objets de domaine et les regles de jeu. `Game` est le moteur central : il gere le plateau (`Board`), le sac (`Bag`), la liste des joueurs, le tour courant, le mode blitz (minuterie par joueur) et l'historique undo/redo. `MoveHandler` est le seul composant autorise a modifier l'etat du jeu en reponse a un `Move`. `Scoring` calcule les points (multiplicateurs de cases, bonus Bingo a 50 points si 7 tuiles posees). `MoveGenerator` genere la liste des coups jouables pour l'IA. `UndoRedo` gere deux piles (`Stack<Move>`) pour l'annulation et le rejeu.

**Couche metier — dictionnaire** Le package `model.dictionary` expose une interface `Dictionary` (methodes `add`, `contains`, `findWordsWithRackAndHook`) implementee par trois structures : `Trie` (recherche prefixe generique), `Dawg` (automate acyclique deterministe, optimise pour la validation rapide) et `Gaddag` (structure bidirectionnelle indispensable pour la generation de coups). Le controleur et l'IA travaillent exclusivement avec `Gaddag`, qui est charge une seule fois depuis le fichier dictionnaire selon la langue active.

**Couche metier — IA** Le package `model.ai` regroupe `AiPlayer` (sous-classe concrete de `Player`, surcharge `isAutonomous()` et `playTurn()`), `MinimaxSolver` (`Minimax` et `Expectiminimax` avec profondeur et limite de temps configurables, 200 tirages de simulation par noeud) et `MlAgent` (agent TensorFlow Java, chargement gracieux : si le modele est absent, l'IA se replie sur le `Minimax` sans crash).

**Couche metier — joueurs** La classe abstraite `Player` (dans `model.interfaces`) unifie `HumanPlayer` et `AiPlayer`. Elle encapsule le nom, le score, le Rack et la minuterie blitz. Les methodes principales sont `enableBlitzClock()`, `startTurnTimer()`, `pauseTurnTimer()` et `isOutOfTime()`. `HumanPlayer` n'ajoute aucune logique : il herite simplement de `Player` sans surcharger `isAutonomous()` (retourne `false` par default). Un joueur reseau est represente cote client par un `Player` standard dont les coups sont pilotes par `GameClient`.

**Couche vue** Le package `view` expose l'interface `UserInterface`. Elle definit quatre methodes : `refresh()`, `displayMessage()`, `displayError()` et `displaySuccess()`. Deux implementations concretes existent : `CliView` (rendu ANSI via `BoardRenderer`, `RackRenderer`, `PlayerRenderer`, `MessageRenderer`) et `JavaFxView` (interface graphique JavaFX avec `BoardPanel`, `RackPanel`, `ScorePanel`, `ControlPanel`, `MessagePanel`). Les launchers `CliLauncher` et `GuiLauncher` dans `view/optionlancement/` sont le point de transition entre App et les vues concretes.

**Persistence** Le package `model.savefiles` contient `ConfigLoader` (lecture/ecriture du fichier `.scrabblerc` au format INI, section `[defaults]`, commentaires `# ignores`, creation automatique si absent), `SaveManager` (serialisation de l'etat de jeu en trois blocs ASCII : `[settings]`, `[game]`, `[history]`) et `GameLoader` (deserialisation et reconstruction d'un `Game` depuis un fichier de sauvegarde).

**Reseau** Le package `model.network` est organise en facade (`NetworkManager`), client (classe `GameClient`) et serveur (classes `GameServer`, `ClientHandler`, `OnlineGame`). `NetworkManager` maintient une liste de `NetworkObserver` (CLI et GUI s'y enregistrent). Il orchestre `GameServer` (TCP 12345, threads par client, liste synchronisee) et `DiscoveryService` (UDP 12346, broadcast de decouverte automatique). Les echanges transitent via `PacketParser`.

**Internationalisation** La classe `I18n` (dans le package `i18n`) est un singleton synchronise backed par `ResourceBundle`. Elle expose `setLanguage(lang)`, `getLanguage()` et `translate(key, args...)`. Deux langues sont supportees : "en" (default) et "fr". La langue conditionne le chargement du dictionnaire et la distribution des lettres dans `Bag`.

## 2.2 Diagramme UML global

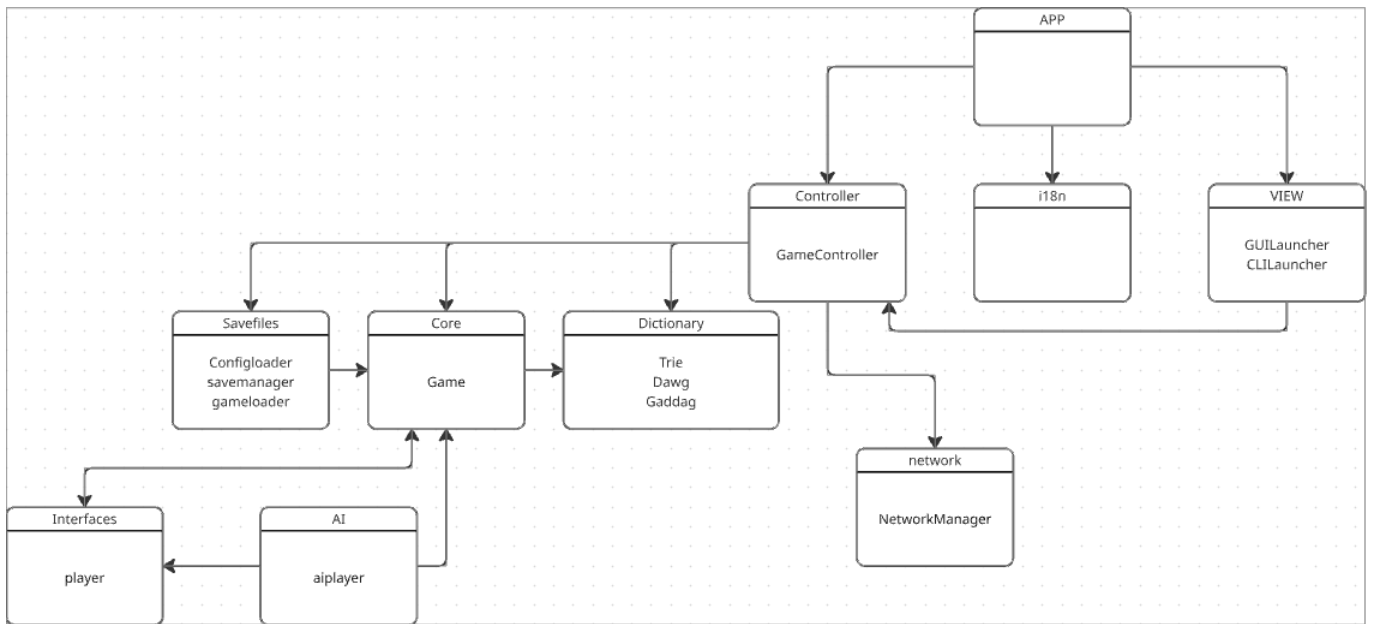


FIGURE 1 – Capture UML globale de l’architecture du projet (source : capture d’écran du diagramme UML).

## 2.3 Principales interfaces et points d’extension

Trois interfaces structurent l’évolutivité du projet.

**UserInterface** — **contrat commun des vues CLI et JavaFX** `CliView` et `JavaFxView` implementent cette interface. `GameController` ne connaît que `UserInterface` — il ignore quelle implementation est active. Ajouter une vue web revient à implementer cette seule interface.

**Dictionary** — **contrat interchangeable entre Trie, Dawg et Gaddag** En pratique, le controleur et l’IA n’instancient que `Gaddag` (qui implemente `Dictionary`). Substituer une autre structure lexicale ne necessite qu’une nouvelle implementation de cette interface.

**Player** — **abstraction commune des joueurs humains et IA** `HumanPlayer` herite directement sans surcharge. `AiPlayer` surcharge `isAutonomous()` (retourne `true`) et `playTurn()` (delegue à `MinimaxSolver`). Le moteur de jeu appelle `player.isAutonomous()` à chaque tour pour decider s’il doit attendre la saisie utilisateur ou decencher le jeu automatique.

**NetworkObserver** — **pattern Observer pour les evenements reseau** `CliNetworkBridge` et `NetworkGameBridge` (cote GUI) implementent cette interface et s’enregistrent aupres de `NetworkManager`. Ajouter un nouveau type de client reseau revient à implementer `NetworkObserver`.

## 2.4 Flux d’exécution principal

Le deroulement d’un tour illustre comment les couches collaborent :

Les dependances sont strictement descendantes : la vue ne connaît que `UserInterface`, le controleur ne connaît que les interfaces `Dictionary`, `UserInterface` et `Player`. `Game` et `MoveHandler` ignorent totalement l’existence de la vue et du reseau.

## 2.5 Gestion des coups et pattern Command

Chaque action de jeu est encapsulee dans un objet `Move` construit par `factory methods` statiques :

`MoveHandler` est le seul executeur : il valide le coup, met à jour `Board`, `Bag` et les scores, puis renseigne les champs mutables de `Move` (positions reellement posees, tuiles piochees, score gagne) pour permettre l’annulation. `UndoRedo` maintient deux `Stack<Move>` : `history` et `redoStack`. `addMove()` vide toujours `redoStack` (nouvelle branche d’historique).

## 2.6 Generation du plateau et pattern Factory

`StandardBoardFactory` applique le pattern `Factory` pour creer le plateau sans que le moteur ne connaisse les coordonnees des bonus :

`SquareType` est une enumeration a cinq valeurs, chacune portant ses multiplicateurs :

Les coordonnees de bonus du plateau 21x21 sont calculees par interpolation lineaire depuis les coordonnees du plateau 15x15 (`scaleCoordinate`), ce qui permet de supporter les deux variantes sans dupliquer les donnees de configuration. `Game` choisit la taille du plateau selon le `GameMode` recu a la construction : `new Board(21)` pour `GameMode.SUPER`, `new Board()` (taille par default 15) pour `GameMode.STANDARD`.

### 3 Algorithmes et Structures de données

Le cœur décisionnel de notre intelligence artificielle repose sur des algorithmes de recherche arborescente. Le défi majeur du Scrabble étant l'information imparfaite (le chevalet adverse est caché), nous avons dû adapter ces structures de données classiques à des environnements probabilistes.

#### 3.1 L'Algorithme Minimax

Historiquement conçu pour des jeux à information parfaite et à somme nulle comme les Échecs, l'algorithme Minimax modélise l'arbre des coups possibles en alternant un tour où l'IA cherche à maximiser son score, et un tour où elle simule un adversaire cherchant à minimiser l'avantage de l'IA.

Pour adapter cet algorithme au Scrabble, notre implémentation déduit d'abord les tuiles restantes, à savoir toutes les tuiles moins celles présentes sur le plateau et dans le chevalet de l'IA. Ensuite, le solveur utilise une méthode de Monte-Carlo : il simule l'adversaire en tirant aléatoirement 200 chevalets virtuels de 7 lettres parmi ces tuiles restantes. Le Minimax évalue alors le pire scénario possible.

Voici comment cette logique pessimiste se traduit dans l'implémentation de notre `MinimaxSolver` :

##### 3.1.1 L'impact crucial de l'heuristique

Au Scrabble, évaluer un coup uniquement par les points qu'il rapporte sur le plateau n'est pas le plus pertinent. L'algorithme intègre donc une fonction heuristique d'évaluation du "reliquat" (`leaveScore`) qui note la qualité des lettres conservées sur le chevalet après un coup. Cette heuristique va attribuer des bonus ou des malus en fonction de la "qualité" du chevalet : Joker (+15.0), 'S' (+8.0), Lettres difficiles (-6.0), équilibre voyelles/consonnes (+5.0), uniquement consonne/voyelle (-12.0).

##### 3.1.2 Complexité, Coût et Performances

Dans notre implémentation de Monte-Carlo, l'algorithme ne dispose pas d'élagage Alpha-Beta. La complexité est donc strictement de  $O(B \times N \times B)$  pour une anticipation de deux plis (Profondeur = 2), où  $B$  représente le facteur de branchement (nombre de coups légaux) et  $N = 200$  le nombre de tirages simulés. Le coût en ressources est massif et identique pour Minimax et Expectiminimax, car tous deux parcourent la même boucle de 200 simulations par coup. Le goulot d'étranglement réside dans l'appel répété au `MoveGenerator` pour chaque scénario adverse. Sur le plan du jeu, la performance défensive du Minimax est excellente : l'IA « verrouille » le plateau pour empêcher l'adversaire de marquer. Cependant, ce pessimisme est à double tranchant : en s'attendant toujours au meilleur tirage adverse, l'IA ferme le jeu, sacrifiant parfois des opportunités offensives pour éviter un risque statistique pourtant faible.

#### 3.2 L'Algorithme Expectiminimax

L'Expectiminimax est l'évolution naturelle du Minimax pour les jeux intégrant du hasard. Il introduit un nouveau type de nœud dans l'arbre de recherche : les "nœuds de chance". Au lieu de considérer que l'adversaire jouera le pire coup absolu, l'algorithme calcule l'espérance mathématique de tous les coups possibles en fonction de leur probabilité d'occurrence.

Comme le montre notre implémentation ci-dessous, la structure de données est identique au Minimax (la même boucle de 200 itérations), mais la méthode d'agrégation change :

##### 3.2.1 L'impact de l'heuristique

En termes simples, l'Expectiminimax ne se contente pas de calculer mathématiquement les risques sur le plateau. Pour juger de la force réelle d'un coup, l'IA additionne le risque statistique sur le plateau et la qualité de sa main évaluée par l'heuristique. L'IA cherche donc toujours l'équilibre parfait entre "limiter la casse probabiliste" et "garder de bonnes lettres pour l'avenir".

##### 3.2.2 Complexité, coût et performances

La complexité est la même ( $O(B \times N \times B)$ ). Bien que son coût soit strictement identique à celui de notre Minimax actuel, l'Expectiminimax a l'inconvénient théorique de ne presque pas pouvoir être optimisé par des techniques d'élagage classiques. Le style de jeu devient plus naturel et audacieux. Dans notre code, son coût temporel étant identique au Minimax Monte-Carlo, l'Expectiminimax s'avère supérieur dans la prise de ses décisions.

### 3.3 Comparaison des arbres de décision

Les schémas ci-dessous mettent en évidence que la différence entre nos deux algorithmes n'est pas structurale (l'arbre exploré est identique), mais réside uniquement dans la méthode mathématique d'agrégation des nœuds de simulation, comme démontré dans nos extraits de code précédents.

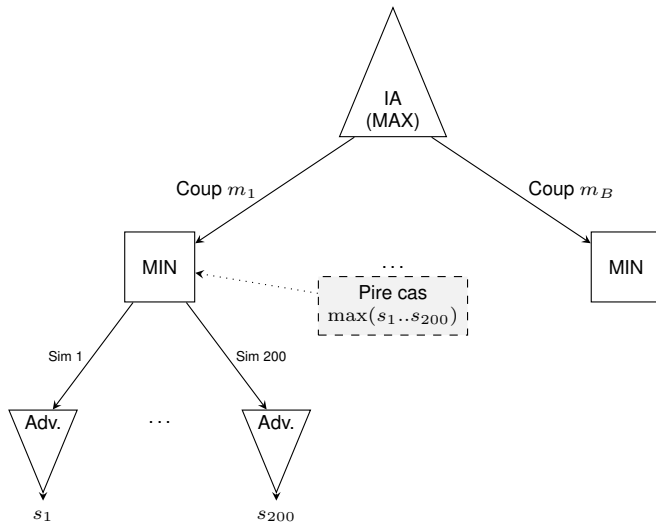


FIGURE 2 – Agrégation Pessimiste

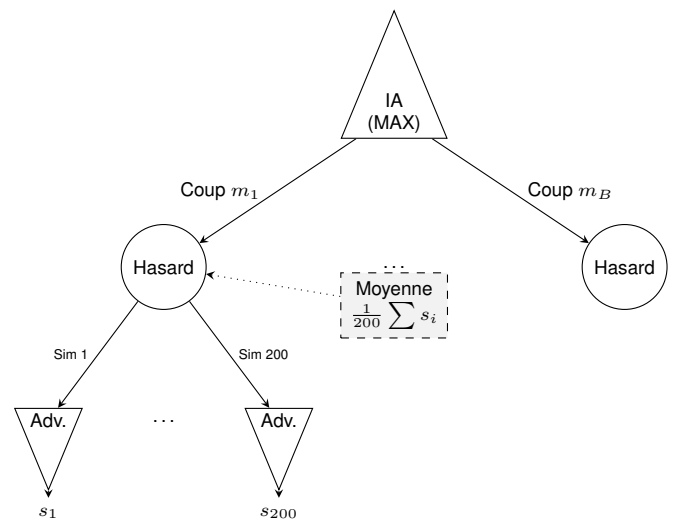


FIGURE 3 – Agrégation Probabiliste

FIGURE 4 – Comparaison des Arbres de Décision

### 3.4 Analyse des performances (Benchmark)

Afin de confronter notre étude théorique à la réalité matérielle, nous avons nous-mêmes mis en place un benchmark interne. Pour ce faire, nous avons instrumenté le code de notre solveur pour extraire des métriques physiques en conditions réelles de jeu. Le chronomètre d'interruption (`timeLimits`) de notre IA était fixé à 5000 millisecondes.

Le tableau ci-dessous synthétise nos résultats, basés sur des échantillons représentatifs de la prise de décision de notre moteur au cours d'une partie complète :

| Algorithme     | Temps   | Statut  | Coups IA Évalués | Simulations Adv. | Moy. Sim/Coup |
|----------------|---------|---------|------------------|------------------|---------------|
| Expectiminimax | 5000 ms | Timeout | 48 sur 84 (57%)  | 9 497            | 197           |
| Expectiminimax | 3575 ms | Complet | 33 sur 33 (100%) | 6 600            | 200           |
| Minimax        | 5001 ms | Timeout | 24 sur 58 (41%)  | 4 680            | 195           |
| Minimax        | 5001 ms | Timeout | 17 sur 41 (41%)  | 3 294            | 193           |
| Minimax        | 2801 ms | Complet | 10 sur 10 (100%) | 2 000            | 200           |

TABLE 1 – Nos résultats expérimentaux (Profondeur 2, SAMPLES\_COUNT = 200)

#### 3.4.1 Interprétation de nos résultats

L'analyse de nos métriques met en évidence trois points cruciaux concernant les limites de notre implémentation :

- **La limite du facteur de branchement** : Nos données prouvent que la capacité de l'IA à évaluer l'arbre complet dépend entièrement de la complexité du plateau à l'instant T. Lorsque le nombre de mots possibles est faible (ex : 10 ou 33 coups), le moteur termine son parcours de Profondeur 2 confortablement avant le temps limite (2.8s et 3.5s). À l'inverse, dès que les choix s'élargissent, l'IA est systématiquement bridée.
- **Le coût de l'aveuglement temporel** : Dans le pire scénario que nous avons mesuré (17 coups évalués sur 41), le *Timeout* force l'IA à jouer en ignorant totalement 59% des opportunités légales, l'empêchant potentiellement de trouver le mot optimal.
- **L'efficacité de notre coupe-circuit** : Notre moteur est capable de simuler un volume massif de plateaux adverses (jusqu'à 9 497 en 5 secondes). Lorsque le *Timeout* est déclenché, la moyenne des simulations tombe légèrement sous la barre des 200. Cela démontre que notre algorithme d'interruption fonctionne avec préci-



sion : il avorte la boucle interne de Monte-Carlo au milieu de l'évaluation pour garantir la fluidité du jeu en temps réel.

Ces mesures empiriques que nous avons relevées confirment notre conclusion théorique : bien que la Profondeur 2 soit mathématiquement atteinte, l'absence d'élagage dans notre approche de Monte-Carlo bride physiquement la rentabilité de l'arbre dès que le facteur de branchement dépasse la trentaine de mots candidats.

### 3.5 Le problème DAWG/GADDAG

La génération de coups légaux au Scrabble impose une contrainte forte : la structure doit, en temps réel, trouver tous les mots jouables à partir d'une lettre déjà posée sur le plateau, que ce soit vers la gauche, la droite, ou les deux. Les structures classiques comme les tableaux de chaînes ou les arbres préfixes naïfs (Trie) sont inadaptées à cette contrainte, soit par leur lenteur de parcours, soit par leur consommation mémoire excessive face à un dictionnaire de 180 000 mots comme ceux que nous avons utilisés.

Deux structures de données académiques répondent à ce problème : le DAWG (Directed Acyclic Word Graph), conçu pour la validation rapide de mots, et le GADDAG, une extension inventée spécifiquement pour le Scrabble par Steven Gordon en 1994 permettant une recherche bidirectionnelle depuis n'importe quelle ancre. Afin de justifier notre choix final, nous avons implémenté et benchmarké les deux structures sur notre dictionnaire.

### 3.6 Le DAWG — Directed Acyclic Word Graph

La structure de données DAWG représente une solution performante alliant vitesse d'exécution et économie de mémoire. Il s'agit d'un automate fini déterministe où chaque chemin partant de la racine correspond à un mot unique du dictionnaire. Sa particularité, qui la distingue d'un arbre préfixe classique, réside dans son optimisation poussée : le DAWG regroupe les mots partageant les mêmes préfixes, mais aussi les mêmes suffixes.

Par exemple, les mots CHAT et PLAT possèdent des débuts différents mais une terminaison identique ; lors de la construction, la structure va donc fusionner ces deux entrées au niveau de leur suffixe commun AT.

Cette approche permet de réduire drastiquement la redondance de l'information tout en garantissant une validation très rapide.

Lexicon:

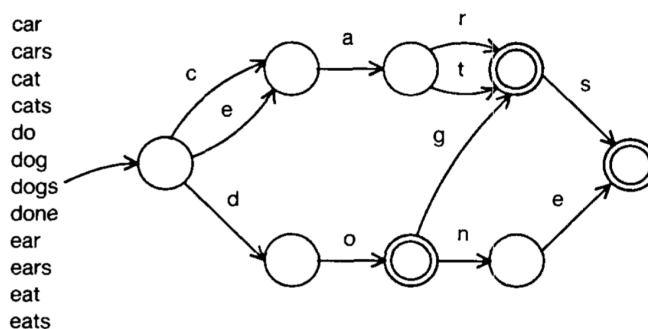


FIGURE 5 – Exemple d'une structure DAWG. (Source : [?])

#### 3.6.1 Construction et complexité

La création d'un DAWG optimisé repose sur l'algorithme de Daciuk [?], qui impose une création des mots selon l'ordre lexicographique. Cette contrainte permet de traiter le dictionnaire de manière séquentielle : lorsqu'un nouveau mot est ajouté, l'algorithme identifie la partie de la structure qui ne sera plus modifiée et procède à la fusion des états équivalents, c'est-à-dire des nœuds possédant les mêmes transitions vers les mêmes états terminaux.

En matière de performance, la construction affiche une complexité temporelle de  $O(|W| \times L)$ , où  $|W|$  représente le nombre total de mots et  $L$  leur longueur moyenne.

Pour la phase de jeu, la recherche d'un mot s'effectue en un temps linéaire  $O(N)$  par rapport à la longueur  $N$  de la chaîne de caractères, ce qui garantit une validation quasi instantanée. Cependant, cette structure unidirectionnelle montre ses limites dès lors qu'une lettre « ancre » est déjà fixée au milieu d'un mot potentiel sur le plateau. Le DAWG ne permettant pas de remonter le chemin vers la racine pour identifier les débuts de mots possibles, le moteur de

jeu est alors contraint d'explorer aveuglément tous les préfixes imaginables avant d'atteindre l'ancre, ce qui rend la génération de coups particulièrement inefficace dans ces configurations.

### 3.6.2 Avantages et limites pour le Scrabble

L'intérêt majeur du DAWG réside dans sa compacité exceptionnelle, permettant de compresser les centaines de milliers de mots d'un dictionnaire en un volume de données souvent inférieur à deux mégaoctets. Cependant, si cette structure est excellente pour la validation d'un mot saisi par un utilisateur, elle s'avère plus limitée pour la phase de génération automatique de coups. En raison de son parcours strictement unidirectionnel, de la racine vers les feuilles, le DAWG ne peut pas explorer efficacement les possibilités de jeu s'articulant autour d'une ancre déjà présente sur la grille.

Pour placer un mot contenant une lettre fixe en son centre ou à sa fin, l'algorithme est incapable de remonter le graphe pour identifier les préfixes compatibles. Il est alors contraint de tester à l'aveugle une multitude de combinaisons de lettres en espérant atteindre l'ancre, ce qui provoque une inefficacité notoire face à l'explosion combinatoire des tirages. Cette incapacité à traiter nativement la recherche bidirectionnelle souligne la nécessité d'une structure plus évoluée, capable de s'affranchir de la contrainte du parcours de gauche à droite : le GADDAG.

## 3.7 Le GADDAG — la solution pour le Scrabble

Le GADDAG, conçu par Steven Gordon en 1994 [?], a donc apporté une réponse définitive au problème de l'ancrage en permettant une recherche bidirectionnelle efficace. Contrairement au DAWG qui ne stocke qu'une version linéaire de chaque mot, le GADDAG génère  $n$  représentations différentes pour un mot de longueur  $n$ , une pour chaque position possible d'une lettre sur le plateau. La structure repose sur une transformation : pour chaque lettre choisie comme ancre, la partie située à sa gauche est inversée, puis séparée de la partie droite par un caractère spécial « > » servant de délimiteur.

À titre d'exemple, le mot « CHAT » est décomposé en quatre représentations distinctes :

1. **C > H A T** (l'ancre est C)
2. **H C > A T** (l'ancre est H : on place C à gauche, puis on complète A-T à droite)
3. **A H C > T** (l'ancre est A : on place C et H à gauche, puis T à droite)
4. **T A H C >** (l'ancre est T : on complète tout le mot vers la gauche)

Cette méthode se généralise sans perte d'efficacité pour des mots plus complexes comme « SCRABBLE », qui produira huit entrées allant de « S>CRABBLE » jusqu'à la forme totalement inversée « ELBBARCS> ». Cette multiplication des chemins garantit que, pour n'importe quelle lettre du mot déjà présente sur la grille, il existe un chemin unique dans l'automate commençant par celle-ci et permettant de créer le reste du mot.

### 3.7.1 Semi-minimisation

L'augmentation du nombre de représentations par mot pourrait laisser craindre une explosion de la consommation mémoire, mais le GADDAG compense ce volume par une technique dite de semi-minimisation. Gordon a démontré que si les préfixes inversés divergent souvent, les suffixes situés après le caractère pivot « > » présentent une redondance massive. La semi-minimisation consiste donc à fusionner tous les états équivalents rencontrés après le délimiteur, exactement comme dans la construction d'un DAWG.

Pour le mot « CARE », les formes « AC>RE » et « RA>CE » voient leurs branches fusionnées avec n'importe quel autre mot du dictionnaire se terminant par « RE » ou « CE ». De même, des mots partageant des suffixes verbaux, comme « MARCHER » et « JOUER », partageront une structure commune pour leurs formes pivotées au-delà du délimiteur (par exemple « RAHC>MER » et « UOJ>ER » où la terminaison « ER » est mutualisée). Cette optimisation rend la structure déterministe et permet de conserver un seul chemin par couple (mot, ancre), tout en maintenant une empreinte mémoire tout à fait acceptable pour les systèmes modernes.

### 3.7.2 Utilisation dans le moteur de jeu

L'intégration du GADDAG au sein du moteur de jeu transforme radicalement la génération de coups en la rendant purement itérative. Pour chaque case « ancre » identifiée sur le plateau, l'algorithme entame un parcours du graphe en deux phases distinctes. Dans un premier temps, il explore les branches correspondant à la partie gauche du mot (le préfixe inversé) en remontant vers les cases vides à gauche ou en haut de l'ancre. Dès qu'il rencontre le

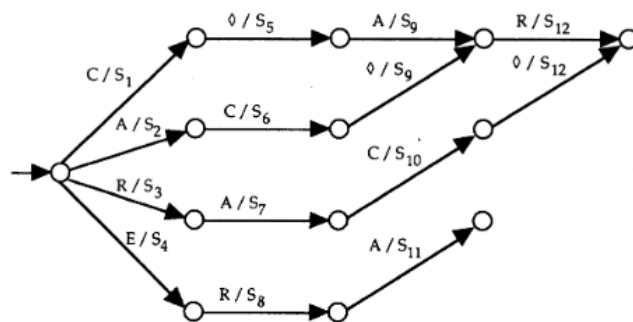


FIGURE 6 – Graphe du GADDAG semi-minimisé pour « CARE ». (Source : [?])

caractère « > » dans l'automate, il bascule dans la seconde phase pour compléter le mot vers la droite ou vers le bas avec les lettres restantes du tirage.

Cette approche permet de découvrir l'intégralité des coups légaux en un seul passage dans l'automate, sans jamais avoir besoin de revenir en arrière ou de deviner une position de départ. En éliminant les tests de préfixes inutiles propres au DAWG, le GADDAG permet à l'IA d'évaluer des milliers de possibilités par seconde, garantissant une réactivité optimale même face à des dictionnaires extensifs et des contraintes de temps réelles.

### 3.8 Analyse du benchmark et justification du choix

TABLE 2 – Benchmark DAWG vs GADDAG — Lexique : 178 691 mots

| Structure | Temps de chargement |
|-----------|---------------------|
| DAWG      | 624 ms              |
| GADDAG    | 1 443 ms            |

| Mot        | Existe | DAWG    | GADDAG   |
|------------|--------|---------|----------|
| APPLE      | vrai   | 3,79 µs | 8,91 µs  |
| ZYZZYVA    | vrai   | 4,32 µs | 9,70 µs  |
| SCRABBLE   | vrai   | 3,96 µs | 8,08 µs  |
| JAVA       | vrai   | 2,75 µs | 8,20 µs  |
| INEXISTANT | faux   | 4,19 µs | 13,94 µs |

| Ancre | Rack    | DAWG     | GADDAG   | Coups (D) | Coups (G) |
|-------|---------|----------|----------|-----------|-----------|
| E     | RSTLNA  | 2,966 ms | 4,752 ms | 170       | 170       |
| S     | CRABBLE | 1,184 ms | 1,288 ms | 112       | 112       |
| Z     | AEIOUST | 0,467 ms | 0,269 ms | 16        | 16        |
| X     | AILNOR  | 0,289 ms | 0,178 ms | 11        | 11        |

Afin de justifier notre choix, nous avons mis en place un benchmark comparatif entre les deux structures, conduit sur notre lexique de 178 691 mots. Nos résultats révèlent une image nuancée qui justifie pleinement le choix du GADDAG comme structure centrale du moteur de jeu..

Le GADDAG accuse un temps de chargement plus élevé (1 443 ms contre 624 ms pour le DAWG), ce qui s'explique directement par sa transformation structurelle : chaque mot de longueur  $n$  génère  $n$  représentations distinctes, alourdissant mécaniquement la phase de construction. De même, sur la validation isolée d'un mot (*contains*), le DAWG reste environ deux fois plus rapide (3 à 4 µs contre 8 à 14 µs). Ces écarts sont cependant sans conséquence

pratique : ces opérations sont réalisées une seule fois à l’initialisation ou de manière ponctuelle, et les valeurs absolues restent négligeables à l’échelle d’une partie.

C’est sur la génération de coups que le choix se justifie pleinement. Les résultats montrent une tendance claire : plus l’espace de jeu est contraint (peu de mots possibles), plus le GADDAG surpasse le DAWG. Pour l’ancre Z avec le rack AEIOUST (16 coups trouvés), le GADDAG est 42 % plus rapide (0,269 ms contre 0,467 ms); pour X avec AILNOR (11 coups), il est 38 % plus rapide (0,178 ms contre 0,289 ms). Ces cas correspondent précisément aux situations les plus fréquentes en fin de partie ou sur des lettres rares, où l’espace combinatoire est restreint mais où chaque coup compte. En fin de partie, le plateau étant déjà largement occupé, les ancres disponibles sont nombreuses mais les extensions possibles réduites : le GADDAG, en naviguant directement depuis chaque ancre sans exploration aveugle des préfixes, évite une explosion combinatoire inutile et identifie les rares coups légaux restants bien plus efficacement que le DAWG, qui continuerait à tester des chemins stériles.

Sur les racks plus généreux (ancre E avec RSTLNA, 170 coups), le DAWG reprend l’avantage. Cela s’explique par le fait que dans ce cas, la quantité de chemins à explorer est si importante que le surcoût structurel du GADDAG se ressent. Toutefois, les deux structures trouvent exactement le même nombre de coups dans tous les cas testés, ce qui confirme la correction de l’implémentation.

|                      | DAWG        |          | GADDAG      |          | Ratio<br>(D/G) |
|----------------------|-------------|----------|-------------|----------|----------------|
|                      | overall     | per move | overall     | per move |                |
| CPU time             |             |          |             |          |                |
| Expanded             | 9:32:44     | 1.344s   | 3:38:59     | 0.518s   | 2.60           |
| Compressed           | 8:11:47     | 1.154s   | 3:26:51     | 0.489s   | 2.36           |
| Page faults          |             |          |             |          |                |
| Expanded             | 6063        |          | 32,305      |          | 0.19           |
| Compressed           | 1011        |          | 3120        |          | 0.32           |
| Arcs Traversed       | 668,214,539 | 26,134   | 265,070,715 | 10,451   | 2.50           |
| Per sec (compressed) | 22,646      |          | 21,372      |          |                |
| Anchors used         | 3,222,746   | 126.04   | 1,946,163   | 76.73    | 1.64           |
| Number of moves      | 25,569      |          | 25,363      |          |                |
| Average score        | 389.58      |          | 388.75      |          |                |

FIGURE 7 – Benchmark original de S.Gordon en 1994. (Source : [?])

À titre de comparaison, Gordon [?] avait mesuré sur 1000 parties aléatoires que le GADDAG réduisait le temps CPU d’un facteur 2,5 environ par rapport au DAWG, avec 1 946 163 ancres traitées contre 3 222 746 pour le DAWG (ratio de 1,64). Bien que l’écart mesuré dans notre benchmark soit moins prononcé, ces résultats confirment l’avantage structurel du GADDAG sur la génération de coups.

**Conclusion du choix.** Le DAWG est une structure excellente pour la validation de mots, mais son parcours strictement unidirectionnel le contraint à explorer aveuglément des préfixes inutiles dès qu’une ancre est présente sur la grille. Le GADDAG, en encodant toutes les positions d’ancrage possibles, élimine ce surcoût algorithmique et offre des performances supérieures précisément là où cela importe le plus : les situations tendues, comme les fins de partie au Scrabble. Le léger surcoût en mémoire et en temps de chargement est un compromis largement acceptable au regard du gain en efficacité durant le jeu.

## Première partie

# Spécifications Étendues

## 4 F21 — Sauvegarde et restauration d’une partie

### 4.1 Description

Le programme doit pouvoir sérialiser une partie en cours dans un fichier texte ASCII et reconstruire une partie complète à partir d’un fichier de sauvegarde. Le parseur doit être robuste aux erreurs de format et les signaler avec précision (type d’erreur et numéro de ligne). L’implémentation repose sur les classes `SaveManager.java` et `GameLoader.java`.

### 4.2 Prérequis

- **F2 — Configuration** : Paramètres par défaut (langue, blitz, etc.) via `.scrabblerc` — `ConfigLoader.java`
- **F10 — Règles du jeu** : État complet du jeu : plateau, scores, joueurs, historique, tour courant
- **F15 — Shell interactif** : Commandes `save FILE` et `load FILE` — `GameControllerAux.java`
- **F22 — Commentaires** : Gestion des commentaires `#` et blocs `{ }` — `GameLoader.java`
- **F23/F24/F25 — Formats** : Sections `[settings]`, `[game]` et `[history]` du fichier de sauvegarde

### 4.3 Type de besoin

Fonctionnel.

### 4.4 Sous-besoin 1. Sauvegarder la partie dans un fichier

**Description** : Sérialiser l’état courant du jeu dans un fichier texte ASCII en écrivant successivement les trois sections obligatoires.

**Fonction associée** :

```
void saveGame(Game game, String filePath) throws IOException
```

**Actions** :

- Ouvrir (ou créer) le fichier au chemin indiqué.
- Ecrire successivement `[settings]`, `[game]` puis `[history]`.
- Fermer le fichier proprement.
- Remonter `IOException` en cas d’échec (gérée par CLI ou GUI).

**Résultat attendu** :

- *Succes* : fichier complet et bien forme écrit sur le disque.
- *Echec* : exception `IOException` remontée, aucun fichier partiel conserve.

**Test unitaire** :

- **Cas 1** : Entree : etat complexe (plateau avec mots, 2 joueurs, scores, racks, coups pass/play). Attendu : fichier cree avec 3 sections, verifie via `SaveManagerTest.java`.
- **Cas 2** : Entree : conversion de coordonnees `a1v / h8h`. Attendu : format des coordonnees correct dans `[history]`.
- **Cas 3** : Entree : partie vide (aucun coup joue). Attendu : plateau serialise en lignes de tirets, section `[history]` vide.

### 4.5 Sous-besoin 2. Serialiser la section `[settings]`

**Description** : Ecrire les parametres globaux de la partie et les parametres IA joueur par joueur sous la section `[settings]`.

**Parametres écrits** :

- `blitz` — mode blitz active ou non.
- `super-scrabble` — plateau  $21 \times 21$ .
- `players-count` — nombre de joueurs.
- `debug / verbose` — niveaux de trace.
- `player-N-type` — human ou ai.
- `player-N-ai-mode` — algorithme IA (ex. : minimax).

- player-N-name — nom du joueur.

**Resultat attendu :**

- Section [settings] complete et coherente avec la configuration de partie.

## 4.6 Sous-besoin 3. Serialiser la section [game]

**Description :** Ecrire l'état du plateau, les racks et les scores de chaque joueur.

**Fonctions internes :**

```
saveBoard(PrintWriter writer, Board board)
serializeRack(Player p)
```

**Format produit :**

- Premiere ligne : index du joueur courant (1-based).
- Plateau ligne par ligne : lettre majuscule ou tiret - pour une case vide.
- rack-N: LETTRES — reserve de lettres du joueur N.
- score-N: valeur — score courant du joueur N.
- language en — langue de la partie (non dans [settings]).

**Test unitaire :**

- **Cas 1 :** Entree : plateau avec mot COUNT en g5h. Attendu : la ligne g contient C O U N T aux bonnes colonnes.
- **Cas 2 :** Entree : plateau entierement vide. Attendu : toutes les cases sont representees par -.
- **Cas 3 :** Entree : joueur 1 score 15 / joueur 2 score 10. Attendu : presence de score-1: 15 et score-2: 10.

## 4.7 Sous-besoin 4. Serialiser la section [history]

**Description :** Ecrire les coups joues dans l'ordre chronologique au format defini par F25.

**Fonctions internes :**

```
saveHistory(PrintWriter writer, Game game, List<Move> history)
readFullWordFromBoard(Board board, Move move)
convertPointToCoord(Point p)
```

**Formats de coup supportes :**

- N pass — le joueur passe son tour.
- N exchange ABC — le joueur echange des lettres.
- N g5h WORD ou N e6v WORD — coup normal horizontal ou vertical.

**Test unitaire :**

- **Cas 1 :** Entree : joueur 1 joue COUNT en g5h, joueur 2 joue PHOTO en e6v. Attendu : presence de 1 g5h COUNT et 2 e6v PHoTO en tete d'historique.
- **Cas 2 :** Entree : aucun coup joue (partie neuve). Attendu : section [history] presente, sans ligne de coup.
- **Cas 3 :** Entree : coup de type pass. Attendu : format N pass ecrit correctement.
- **Cas 4 :** Entree : coup de type exchange. Attendu : format N exchange LETTRES ecrit correctement.

## 4.8 Sous-besoin 5. Charger une partie depuis un fichier

**Description :** Parser le fichier de sauvegarde et reconstruire un objet Game complet (joueurs, plateau, racks, scores, tour courant et historique).

**Fonction associee :**

```
Game loadGame(String filePath) throws Exception
```

**Actions :**

- Prelecture du fichier pour detecter le mode super-scrabble et creer le bon plateau (15×15 ou 21×21).
- Lecture sequentielle et parsing des sections [settings], [game] et [history].
- Reconstruction des joueurs humains et IA avec leurs parametres.

- Reconstruction de l'historique complet des coups.

#### Gestion d'erreurs :

- Toute erreur de parsing est remontée sous la forme `Format error at line X: <detail>`.
- La CLI et la GUI affichent ensuite un message utilisateur lisible.
- L'état courant du jeu n'est jamais écrasé en cas d'échec.

#### Test unitaire :

- **Cas 1** : Entree : fichier valide et complet. Attendu : Game non nul avec plateau, scores et historique cohérents (`GameLoaderTest.java`).
- **Cas 2** : Entree : plateau 21×21 (super-scrabble). Attendu : détection correcte et création du bon plateau.
- **Cas 3** : Entree : fichier contenant `score-1: abc`. Attendu : exception `Format error at line X: ....`
- **Cas 4** : Entree : section `[game]` manquante. Attendu : exception avec message explicite.
- **Cas 5** : Entree : coup `exchange` dans l'historique. Attendu : coup restaure correctement.
- **Cas 6** : Entree : fichier inexistant. Attendu : exception fichier introuvable.

## 4.9 Sous-besoin 6. Parser et ignorer les commentaires

**Description** : Supprimer les commentaires avant le parsing tout en conservant une numérotation de ligne fiable pour les messages d'erreur, conformément à F22.

#### Fonction interne :

```
processLineComments(String line)
```

#### Comportement :

- Supprime tout ce qui suit un `#` sur une ligne (commentaire en ligne).
- Ignore les blocs `{ ... }` potentiellement multi-lignes via un état `isInBlockComment`.
- Continue le parsing ligne par ligne en maintenant le compteur de lignes.

#### Test unitaire :

- **Cas 1** : Entree : `debug=true # activer le debug`. Attendu : `debug=true` après suppression du suffixe commentaire.
- **Cas 2** : Entree : `bloc { commentaire\nsur 2 lignes }` puis `language=fr`. Attendu : bloc supprimé, `language=fr` conserve.
- **Cas 3** : Entree : fichier sans commentaire. Attendu : contenu strictement identique en sortie.

## 4.10 Intégration dans l'architecture du projet

| Module           | Responsabilité                                                | Détail                                                                                                                                                                             |
|------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| savefiles        | <code>SaveManager.java</code><br><code>GameLoader.java</code> | <code>SaveManager</code> : sérialisation de la partie.<br><code>GameLoader</code> : parsing et reconstruction du Game.                                                             |
| CLI              | <code>GameControllerAux.java</code>                           | <code>save FILE</code> → <code>SaveManager.saveGame</code><br><code>load FILE</code> → <code>GameLoader.loadGame</code><br>Erreurs affichées via <code>cliView.displayError</code> |
| GUI              | <code>ScrabbleGui.java</code>                                 | Menu <i>Save</i> → <code>SaveManager.saveGame</code><br>Menu <i>Load</i> → <code>GameLoader.loadGame</code><br>Rebind du contrôleur et rafraîchissement UI après chargement.       |
| Configuration F2 | <code>ConfigLoader.java</code>                                | Lit <code>.scrabblerc</code> dans le HOME.<br>Note : le chemin par défaut de sauvegarde n'est pas encore pris en charge si <code>FILE</code> est absent.                           |

## 4.11 Scénario utilisateur

1. L'utilisateur joue plusieurs coups en CLI ou via l'interface graphique.
2. En CLI, il tape : `save ma_partie.scrabble`
3. `SaveManager` écrit `[settings]`, `[game]` puis `[history]` dans le fichier.
4. Le shell affiche un message de succès à l'utilisateur.
5. L'utilisateur relance le programme et tape : `load ma_partie.scrabble`
6. `GameLoader` reconstruit le `Game` : joueurs, plateau, tour courant et historique.
7. Si une ligne est invalide, le programme affiche `Format error at line X: ...` et n'écrase pas l'état courant.

## 5 Bilan critique

L'élaboration de ce prototype de Scrabble a permis de valider la faisabilité technique d'un moteur de jeu complet couplé à des intelligences artificielles complexes. Néanmoins, avec le recul nécessaire sur le cycle de développement, plusieurs limitations architecturales, ergonomiques et algorithmiques ont été identifiées. Cette section dresse un bilan critique du projet et propose des axes d'amélioration.

### 5.1 Architecture Logicielle

Bien que le moteur de jeu (`core`) soit pleinement fonctionnel, son architecture actuelle présente un potentiel de refactoring important. Au fil du développement, un certain couplage s'est créé entre la logique métier (règles du jeu, gestion du plateau), les modules d'intelligence artificielle et l'interface utilisateur. Pour garantir la pérennité et la scalabilité du projet, une restructuration globale serait bénéfique :

- **Découplage et Modularité** : L'application stricte des principes de conception (comme l'injection de dépendances et le pattern MVC) permettrait de rendre les modules totalement indépendants. L'IA ne devrait interagir avec le moteur que via une API interne stricte.
- **Optimisation du MoveGenerator** : Actuellement très sollicité, le générateur de mouvements gagnerait à être optimisé (par exemple via du multithreading pour le parcours du GADDAG), afin de soulager le processeur lors des phases de réflexion de l'IA.

### 5.2 Ergonomie et Interface Utilisateur (GUI)

L'interface graphique, bien que stable, souffre de plusieurs lacunes ergonomiques qui altèrent la fluidité de l'expérience utilisateur (UX). Le système d'interaction avec les tuiles manque de naturel.

- **Absence de Drag-and-Drop inversé** : L'exemple le plus marquant est la gestion des erreurs de placement. Pour retirer une lettre du plateau, le joueur est contraint d'utiliser un bouton global "Annuler le placement" (*Cancel placement*). Une interaction au glisser-déposer (*drag-and-drop*) permettant de remettre directement une tuile spécifique sur le chevalet serait beaucoup plus intuitive.
- **Feedbacks visuels** : Ces rigidités d'interaction cassent le dynamisme naturel d'une partie de Scrabble physique. Des améliorations mineures, telles que des animations de placement plus fluides ou un retour visuel immédiat (surbrillance) lorsqu'un mot formé est invalide avant même sa soumission, moderniseraient considérablement l'application.

### 5.3 Limitations de l'Internationalisation (i18n)

À ce stade, le jeu ne supporte officiellement que deux langues : le français et l'anglais. Cette limitation n'est pas inhérente au moteur de jeu, mais plutôt à la gestion codée en dur des ressources (dictionnaires et distribution des lettres). Le système de chargement devrait être rendu totalement agnostique. L'ajout d'une nouvelle langue ne devrait nécessiter qu'un simple ajout de fichier de configuration (JSON/XML) définissant l'alphabet, la valeur des lettres, leur fréquence d'apparition et le fichier lexique associé, sans aucune recompilation du code source.

### 5.4 Limites des IA Implémentées

Le défi majeur de ce projet résidait dans l'implémentation de l'intelligence artificielle face au problème de l'information imparfaite. Les approches développées ont chacune révélé des limites matérielles ou logiques :

- **Approche Monte-Carlo (Minimax / Expectiminimax)** : Nos mesures empiriques ont prouvé que la complexité temporelle de ces algorithmes provoque un goulot d'étranglement majeur. L'absence de techniques d'élagage (comme l'Alpha-Beta) oblige le moteur à interrompre la recherche via un *timeout* (5 secondes) avant d'avoir pu explorer l'arbre de profondeur 2 de manière exhaustive.
- **Le Machine Learning (TensorFlow)** : L'approche connexionniste offre des performances foudroyantes en temps d'inférence pour trouver des anagrammes. Cependant, l'architecture de notre tenseur d'entrée limite la



vision du réseau neuronal au seul chevalet du joueur. Sans connaissance de la topologie du plateau, ce modèle est incapable de jouer de manière autonome.

## Références

- [1] Andrew W. Appel and Guy J. Jacobson. The world's fastest scrabble program. *Commun. ACM*, 31(5) :572–578, May 1988.
- [2] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [3] Jan Daciuk, Bruce W. Watson, Stoyan Mihov, and Richard E. Watson. Incremental construction of minimal acyclic finite-state automata. *Comput. Linguist.*, 26(1) :3–16, March 2000.
- [4] Steven A. Gordon. A faster scrabble move generation algorithm. *Softw. Pract. Exper.*, 24(2) :219–232, February 1994.